Project: Student Wellness App
Students: Ananya Iyer, Shreyas Ghosh Roy, Jasraj Baweja, Sean Rogers

What We Changed and Why

When we first submitted our project, the website was a clean, well-structured static site with four pages — a home page, a features overview, an about page, and a contact page. The site communicated the concept of the Student Wellness Tracker clearly, but it had no real interactivity. The contact form existed but submitting it did nothing meaningful — it displayed a one-line confirmation with the user's name and nothing else. Three of the four pages had no JavaScript at all, and none of the wellness tracking features described on the site actually worked.

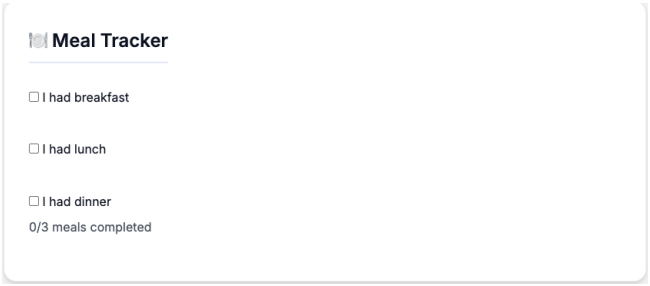
Main Page

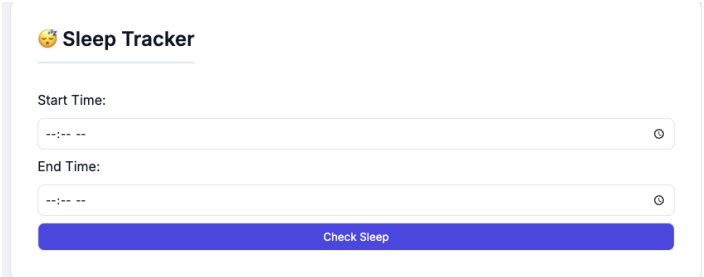
The updated index.html transforms the homepage from a static information page into an interactive student wellness dashboard. The original hero and feature sections remain, but three new working tracker sections were added.

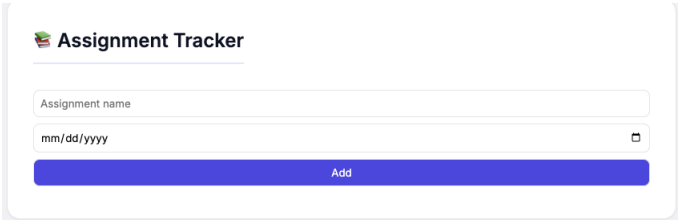
The Meal Tracker includes checkboxes for breakfast, lunch, and dinner with a live progress counter and a congratulatory message when all meals are completed. The checkbox states are saved using localStorage.

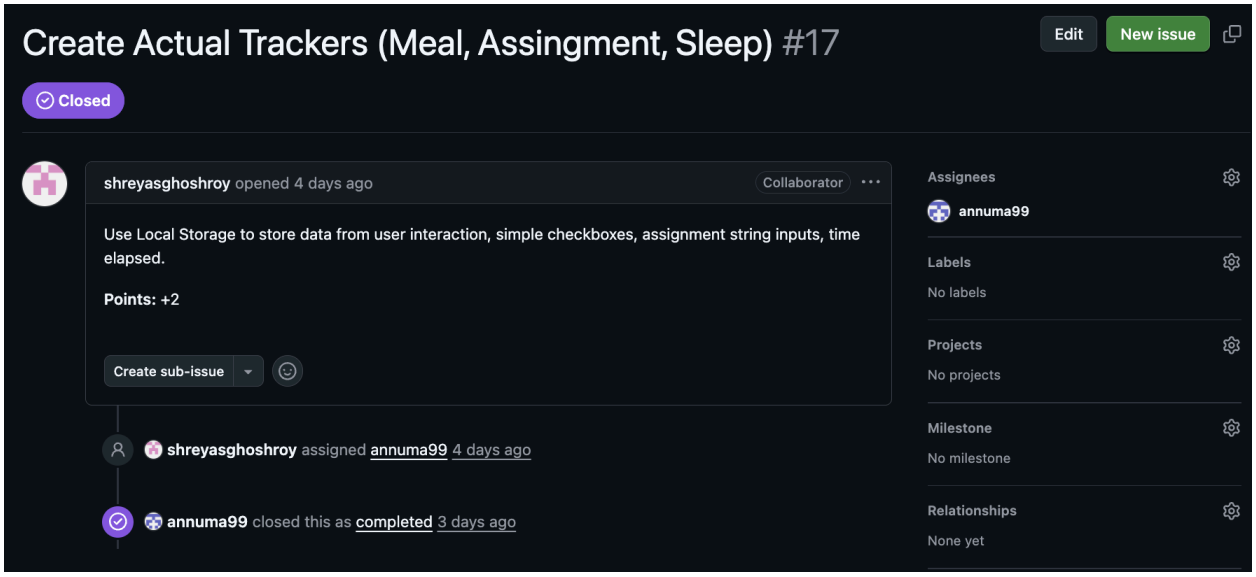
The Assignment Tracker allows users to add assignments with due dates, display them in a list, and mark them as completed with a strikethrough effect. All assignments are stored in localStorage so they persist after refreshing.

The Sleep Tracker lets users enter sleep and wake times, calculates total hours slept, correctly handles sleeping past midnight, and displays a message if the user gets at least eight hours of sleep. The most recent result is also saved in localStorage. This was completed by Ananya.



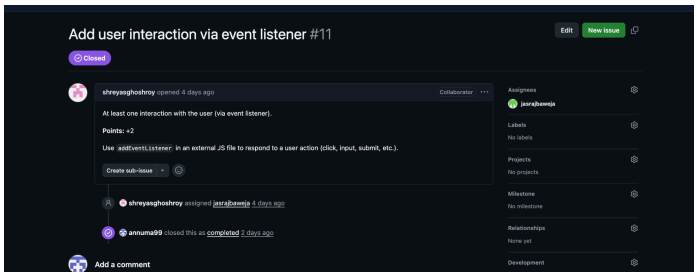


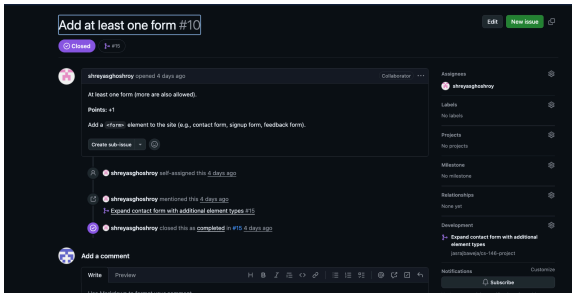




Contact Page

The form was designed to meet the assignment requirement of at least three different types of form elements, and we exceeded that significantly. The final form includes a text input for the user's name, an email input for their address, a select dropdown for choosing a subject category, a set of radio buttons for indicating preferred contact method, a checkbox for opting into a wellness tips newsletter, and a textarea for the message body.



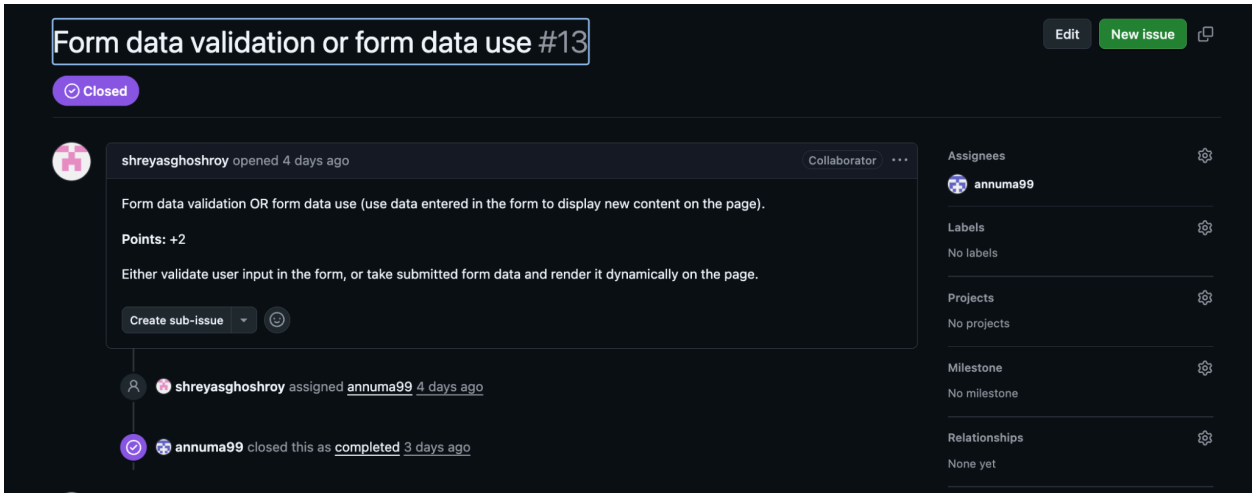


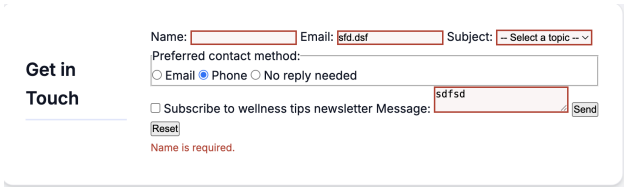
Validation Logic

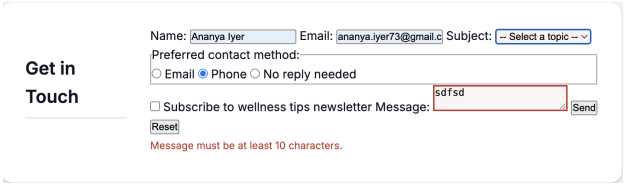
We rewrote main.js from scratch. It now contains dedicated validation functions for each field. The name field checks that something was entered and that it's at least two characters long. The email field checks that it's not empty and matches a standard email pattern. The subject dropdown checks that the user actually selected a topic rather than leaving it on the placeholder. The message field checks that it's not empty and has at least ten characters.

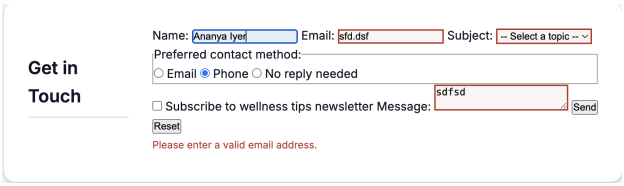
Each of these validators runs in real time as the user fills out the form — on blur for the text fields and on change for the dropdown — so errors appear immediately rather than all at once when the user tries to submit. When a field has an error, a red message appears beneath it and the field gets a red border. When the user fixes the issue, the error disappears. This makes the experience feel responsive and helpful rather than frustrating.

When the user clicks submit, all four validators run one more time. If anything fails, the form doesn't go anywhere and the first invalid field is focused automatically so the user knows where to look. Only when everything passes does the form actually submit. This was completed by Ananya.



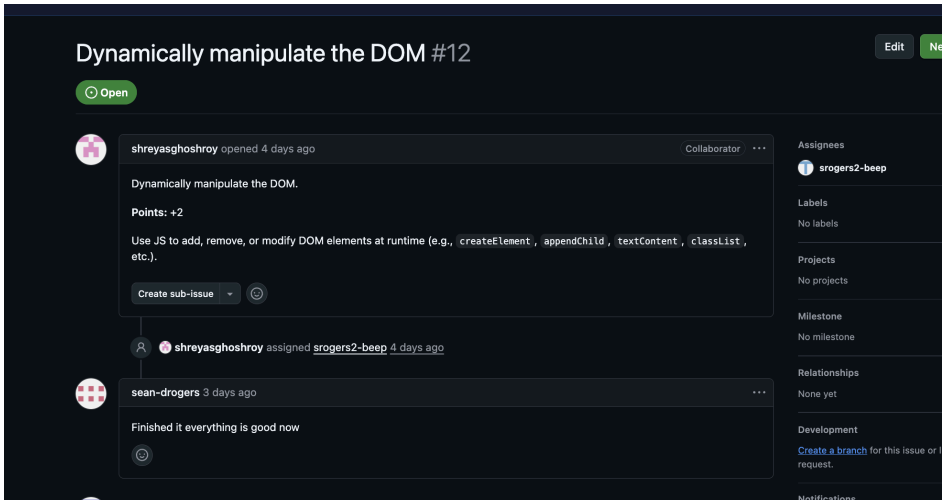


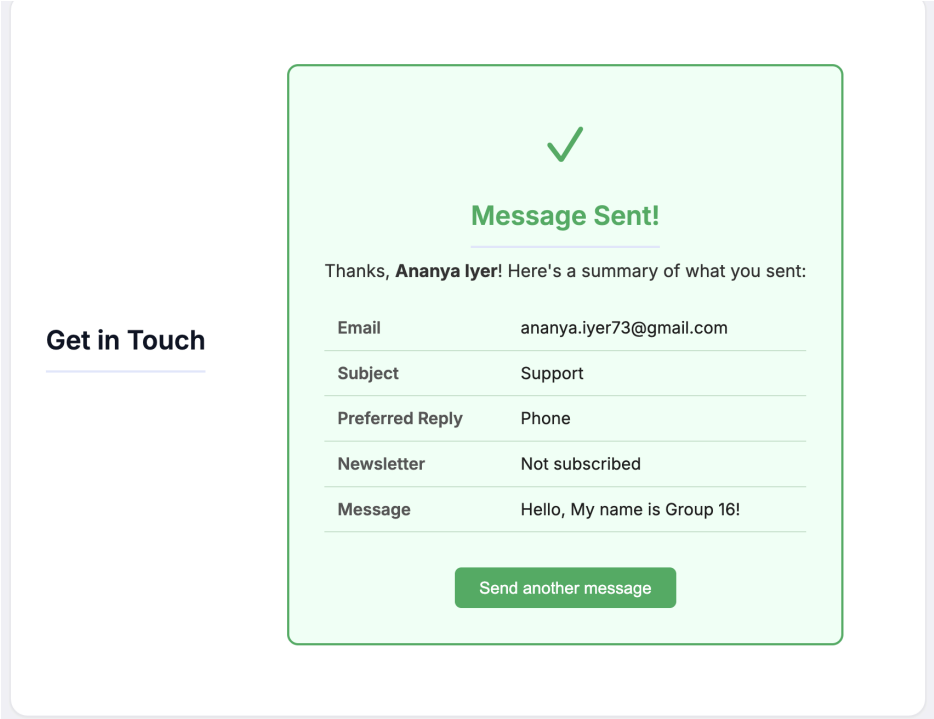




Confirmation Card

On a successful submission, instead of a plain line of text, the form is now replaced with a styled confirmation card that shows the user a full summary of what they sent — their name, email, subject, preferred contact method, whether they subscribed to the newsletter, and their message. This makes the interaction feel complete and trustworthy, especially since there's no real backend sending a confirmation email. This was completed by Sean.





Security

One vulnerability we specifically addressed was XSS — Cross-Site Scripting. Because we're taking user input and displaying it back on the page, there was a risk that someone could type something like <script>alert('hacked')</script> into the name or message field and have it execute as real JavaScript. We fixed this by writing an escapeHTML() utility function that converts all user input into safe text using the browser's own createTextNode() method before

anything gets inserted into the DOM. This was completed by Sean.

```
// --- Utilities ---
function escapeHTML(str) {
  const div = document.createElement("div");
  div.appendChild(document.createTextNode(str));
  return div.innerHTML;
}
```